Ephemeral Meta-Weights for Bandwidth-Aware Transformer Feed-Forward Networks

Anonymous Authors

April 2026

Abstract

Transformer inference and training increasingly run into a weight-bandwidth limit: feed-forward network (FFN) projections are large, reused at every token, and repeatedly streamed from off-chip memory. We study MWG-EW, a neural architecture that replaces persistent FFN matrices with conditional low-rank meta-weights. A small generator produces rank-r descriptors for the current layer and token group; a fused associative kernel immediately consumes the descriptors as $(XU_r)V_r$ and discards them. The dense FFN matrix is never stored or synchronized as a trainable object in the transformed block.

This paper makes three contributions. First, we define a generator-conditioned Transformer FFN that preserves differentiability while exposing rank, token reuse, and basis-bank size as architecture knobs. Second, we give an infrastructure contract for ephemeral descriptor execution and derive bandwidth, latency, memory, and gradient-communication scaling laws. Third, we provide an AI3/Huanxin Ascend 910B2 evaluation harness that runs on all visible NPUs and reports wall-clock latency, traffic estimates, and communication volume for 8B-scale FFN geometry. On four Huanxin AI3 Ascend 910B2 NPUs, the current PyTorch-NPU prototype at $r = 128$ reduces descriptor traffic by 24.9x and trainable FFN state by 18.4x, improves the measured training step by 1.52x, and reduces measured all-reduce latency by 9.1x. The same experiment also shows that the unfused forward path is still slower than dense execution, making kernel fusion and no-write-back enforcement a precise systems target rather than an afterthought. The central claim is not that static low rank is enough; it is that learned conditional generation plus a no-write-back operand lifecycle can make parameter movement an architectural design variable.

1 Introduction

The standard Transformer block stores FFN projections as persistent dense matrices and streams them during every forward pass. For autoregressive decoding, the arithmetic per token can be small relative to the bytes fetched from high-bandwidth memory. For distributed training, the same persistent matrices induce equally persistent gradient synchronization traffic. This paper asks whether a Transformer can improve this operating point by changing the lifecycle of FFN weights rather than only changing their datatype.

MWG-EW turns a subset of FFN weights into generated, short-lived descriptors. The descriptor is not a compressed checkpoint artifact; it is an operand created by a neural generator, consumed by a kernel, and then overwritten. The architecture is therefore both algorithmic and infrastructural: the learning rule determines which descriptors are useful, while the runtime determines whether the descriptors actually avoid off-chip traffic.

This distinction matters because many efficient-model methods still leave the same lifecycle in place: a parameter tensor is stored, loaded, differentiated, and synchronized as a durable object.

MWG-EW studies the complementary question: what happens if a large projection is a transient computation product rather than a resident model object? If this view holds under full training and quality validation, it suggests a new axis for scaling Transformers: not only fewer parameters or fewer bits per parameter, but shorter parameter lifetimes.

The proposal is close in spirit to hypernetworks [2], low-rank adaptation [3], and IO-aware kernels such as FlashAttention [1], but it combines different constraints. Hypernetworks usually materialize generated weights as ordinary model tensors. LoRA stores low-rank factors as persistent adapter parameters. FlashAttention optimizes attention activation traffic. MWG-EW instead treats generated FFN factors as volatile kernel operands.

Contributions. We make the following contributions:

- A Transformer FFN replacement in which a learned generator emits conditional rank-r descriptors for the up, gate, and down projections.
- A no-write-back descriptor lifecycle that connects the neural architecture to memory traffic and distributed synchronization.
- Scaling laws for parameter bytes, descriptor bytes, latency in the memory-bound regime, KV-cache capacity recovery, and all-reduce volume.
- A distributed Ascend NPU benchmark that uses all visible AI3 NPUs and records environment, latency, and communication evidence in machine-readable form.
- A negative static-SVD baseline showing why the method should be judged as conditional generation and lifecycle control rather than ordinary low-rank compression.

2 Architecture

2.1 Baseline Gated FFN

A gated Transformer FFN maps hidden states $X \in \mathbb{R}^{B\times S\times d}$ to

$$Y = (\phi(XW_{\text{gate}}) \odot XW_{\text{up}})\, W_{\text{down}}, \tag{1}$$

where $W_{\text{up}}, W_{\text{gate}} \in \mathbb{R}^{d\times m}$, $W_{\text{down}} \in \mathbb{R}^{m\times d}$, and ϕ is typically SiLU. With fp16 weights, one block stores approximately $6dm$ bytes when m dominates d.

2.2 Ephemeral Meta-Weight FFN

MWG-EW replaces each projection matrix by a conditional descriptor pair:

$$(U_r^{(p)}, V_r^{(p)}) = G_\theta(c_t, \ell, p), \qquad XW_p \approx (XU_r^{(p)})V_r^{(p)}, \tag{2}$$

where $p \in \{\text{up}, \text{gate}, \text{down}\}$, c_t is a token-group context, ℓ is the layer id, $U_r \in \mathbb{R}^{d_{\text{in}}\times r}$, and $V_r \in \mathbb{R}^{r\times d_{\text{out}}}$. The FFN becomes

$$\widetilde{Y} = \Big(\phi((XU_r^{(g)})V_r^{(g)}) \odot (XU_r^{(u)})V_r^{(u)}\Big)\, U_r^{(d)}V_r^{(d)}. \tag{3}$$

We use a basis-bank generator:

$$\alpha = \mathrm{softmax}(A_\ell h_\theta(c_t)), \tag{4}$$

$$U_r^{(p)} = \sum_{k=1}^{K} \alpha_k U_k^{(p)} + \Delta U_\theta^{(p)}(c_t, \ell), \tag{5}$$

$$V_r^{(p)} = \sum_{k=1}^{K} \alpha_k V_k^{(p)} + \Delta V_\theta^{(p)}(c_t, \ell). \tag{6}$$

The bank gives stable reusable directions; the residual head gives input-conditioned adaptation. Practical kernels may generate only rank-channel scales or tile-local residuals when full descriptor synthesis is too expensive.

2.3 Descriptor Lifecycle

The runtime contract is simple and testable:

1. Read token context c_t and layer id ℓ.
2. Generate descriptor factors (U_r, V_r) for each FFN projection.
3. Store the descriptors in an on-chip scratch region.
4. Compute each projection with the associative schedule $(XU_r)V_r$.
5. Release or overwrite the scratch region before the next token group.
6. Synchronize gradients only for generator and basis-bank parameters.

The important distinction is not only low rank. A low-rank factor stored as a long-lived model tensor still creates persistent parameter traffic. The ephemeral contract allows a runtime to avoid writing generated descriptors back to off-chip memory and allows distributed training to synchronize only the generator-side state.

3 Analysis

3.1 Traffic Scaling

For a gated FFN with three projections and fp16 elements, dense weight traffic per block is

$$\mathcal{T}_{\mathrm{dense}} = 6dm. \tag{7}$$

Ignoring the small generator context, rank-r descriptor traffic is

$$\mathcal{T}_{\mathrm{mwg}} = 2r(d+m) + 2r(d+m) + 2r(m+d) = 6r(d+m). \tag{8}$$

The idealized traffic ratio is therefore

$$\Gamma(r) = \frac{\mathcal{T}_{\mathrm{mwg}}}{\mathcal{T}_{\mathrm{dense}}} = \frac{r(d+m)}{dm}. \tag{9}$$

At $d = 4096$, $m = 14336$, and $r = 128$, this gives $\Gamma = 0.0402$, or a 24.9x reduction in descriptor bytes.

Proposition 1 (Memory-bound speedup condition)**.** *Assume per-block latency can be approximated by $L = L_0 + \kappa\mathcal{T}$ with fixed launch/compute term L_0 and bandwidth coefficient $\kappa > 0$. Then* MWG-EW *improves latency whenever $\mathcal{T}_{\mathrm{mwg}} < \mathcal{T}_{\mathrm{dense}}$, and the asymptotic speedup approaches $1/\Gamma(r)$ as L_0 becomes small relative to memory traffic.*

Table 1: Static SVD baseline on 18 pretrained FFN matrices. Low ranks are excellent for bytes but weak as fixed approximations, motivating learned conditional generation.

Rank r	Mean energy	Min energy	Mean cosine
32	7.17%	4.27%	0.266
64	11.98%	8.16%	0.345
128	20.12%	15.46%	0.448
256	33.79%	28.65%	0.582
512	55.61%	50.75%	0.748
1024	85.13%	82.42%	0.925

3.2 Gradient Communication

In data-parallel training, dense FFN gradients require all-reduce traffic proportional to $6dm$ bytes per block. MWG-EW synchronizes generator and descriptor-bank parameters, whose leading term is $6r(d+m)$ plus the small MLP state. The same ratio $\Gamma(r)$ controls the dominant gradient-byte reduction. This is orthogonal to optimizer sharding and tensor parallelism: those systems can still be used on the remaining generator parameters.

3.3 Approximation

Let $\widetilde{W}_p = U_r^{(p)} V_r^{(p)}$ be the generated approximation of W_p. If $\|W_p - \widetilde{W}_p\|_F \le \epsilon_p$, then for bounded activations and L_ϕ-Lipschitz nonlinearity,

$$\|Y - \widetilde{Y}\|_F \le C_1 L_\phi \|X\|_F (\epsilon_{\text{gate}} + \epsilon_{\text{up}}) + C_2 \|X\|_F \epsilon_{\text{down}}, \tag{10}$$

where C_1 and C_2 depend on activation bounds inside the FFN. This connects rank selection to output distortion, while leaving room for end-to-end distillation to improve beyond static SVD.

3.4 Static Low-Rank Is Not the Claim

To separate MWG-EW from ordinary compression, we analyze a static truncated-SVD baseline on 18 FFN matrices from a Qwen-1.5B-family checkpoint. Table 1 reports mean retained singular-value energy and reconstruction cosine. At ranks that are attractive for bandwidth, static SVD preserves too little FFN structure to be a complete solution. This is evidence for the generator side of MWG-EW: the low-rank factors should be conditional operands, not merely fixed compressed weights.

4 Systems Realization

An implementation needs three pieces:

- **Generator placement.** The generator should run adjacent to the FFN kernel and emit descriptor tiles directly into scratch memory.
- **Associative execution.** The kernel computes XU_r followed by ZV_r, fusing activation and gate operations where possible.
- **Lifecycle enforcement.** Descriptor buffers are marked volatile; runtime instrumentation checks that descriptor write-back does not occur.

Table 2: Measured 8B-scale FFN execution on four AI3 Ascend 910B2 NPUs. The current prototype is a PyTorch-NPU implementation of associative execution, not yet a fused no-write-back kernel.

Method	Rank	Fwd ms	Fwd speedup	Train ms	Train speedup
Dense FFN	–	0.9275	1.000x	5.4483	1.000x
MWG-EW	64	1.3920	0.666x	3.6486	1.493x
MWG-EW	128	1.5094	0.614x	3.5931	1.516x
MWG-EW	256	1.2347	0.751x	3.8407	1.419x

The benchmark in this folder measures the associative path and the distributed communication effect. A production kernel should additionally expose hardware counters for off-chip descriptor reads and writes.

5 Experimental Protocol

Hardware. The large-scale target is Huanxin AI3 with Ascend 910B2 NPUs. The launcher uses all visible NPUs through `torch.distributed.run` and records the resolved device count, device names, memory, PyTorch version, and torch-npu version.

Geometry. The default large setting is an 8B-scale FFN block with $d = 4096$, $m = 14336$, decode shape $B = 1, S = 128$, and training shape $B = 2, S = 512$. The rank sweep is $r \in \{64, 128, 256\}$ by default.

Smoke-to-large path. The benchmark now has two remote presets. `ai3_smoke` uses $d = 512, m = 1408$ and short iteration counts to validate the Huanxin session, device stack, output writing, and all-reduce path. `ai3_large` then uses the 8B-scale geometry. This prevents expensive all-NPU runs from being the first check after authentication or environment drift.

Metrics. We report forward latency, backward latency, throughput, trainable parameter bytes, estimated descriptor traffic, dense-to-MWG-EW speedup, all-reduce latency, all-reduce bytes, and KV-cache memory recovered under a fixed budget.

6 Numerical Evidence

Evidence status. The results below were measured on Huanxin AI3 on 2026-04-30 using four Ascend 910B2 NPUs, `torch.distributed.run`, fp16 tensors, and logical device ids `0,1,2,3`. The benchmark uses the 8B-scale FFN geometry $(d, m) = (4096, 14336)$, decode shape $(1, 128, 4096)$, and training shape $(2, 512, 4096)$. The main run used four warmup iterations, twelve forward timing iterations, four training iterations, and four all-reduce iterations. These measurements establish systems behavior for the associative low-rank path and distributed communication. They do not yet establish final language-model quality.

Interpretation. The strongest current operating point for training is $r = 128$: it reduces descriptor traffic from 336.0 MiB to 13.5 MiB, reduces trainable FFN state from 336.0 MiB to 18.254 MiB after generator overhead, improves the measured training step from 5.4483 ms to 3.5931 ms, and reduces measured all-reduce latency from 10.9841 ms to 1.2116 ms. The forward-only path is slower

Table 3: Measured state size, descriptor traffic, and gradient communication for one 8B-scale FFN block on the same four-NPU AI3 run. The all-reduce column reports measured latency for reducing a buffer with the corresponding trainable state size.

Method	Rank	Params MiB	Descriptor MiB	Traffic red.	AllReduce ms
Dense FFN	–	336.000	336.000	1.000x	10.9841
MWG-EW	64	11.127	6.750	49.778x	0.9714
MWG-EW	128	18.254	13.500	24.889x	1.2116
MWG-EW	256	32.507	27.000	12.444x	1.5495

than the dense baseline in the longer run because the prototype executes low-rank factors as unfused PyTorch operations and still materializes intermediate activations. This negative result is useful: it identifies fused descriptor generation, on-chip factor consumption, and no descriptor write-back as the systems requirements that separate MWG-EW from an ordinary low-rank module.

7 Discussion

MWG-EW is most useful when FFN weight traffic dominates and when the target hardware has enough arithmetic headroom to spend extra low-rank multiplies or generator work. It is less attractive when the model is already compute-bound, when rank must approach full dimension, or when the generator is placed so far from the FFN kernel that descriptor traffic reappears.

The architecture also changes the meaning of compression. The goal is not only to reduce checkpoint size, but to change where parameters live during execution. That distinction is why the method belongs partly to neural architecture design and partly to model-runtime infrastructure.

Current limitations. Three gaps are intentionally left visible. First, the measurements are single-block systems evidence; full end-to-end language quality still needs a stable distillation or training run. Second, descriptor lifecycle claims should be confirmed with hardware counters for off-chip reads and writes, not only with software-side byte estimates. Third, the current generator and associative path are deliberately small and benchmarkable; stronger generators and fused kernels may improve both quality and forward latency, but must be measured against their own placement and traffic costs.

8 Conclusion

This paper reframes Transformer FFN weights as generated ephemeral operands. By replacing persistent dense matrices with conditional rank-r descriptors and an associative execution path, MWG-EW reduces both memory traffic and distributed gradient communication. The AI3 benchmark included with this paper is designed to validate the method on all available Ascend NPUs and to produce reproducible evidence for the next draft revision.

References

[1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[2] David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.